Condition Handling

1. Condition Handling – Kompakte System-Referenz für KI-Modelle
2. Grundprinzip
3. Placeholder Resolution
4. Equality
5. Inequality
6. String-Vergleiche
7. Numerisch aussehende Werte
8. CONTAINS
9. AND
10. OR
11. NOT
12. Klammern
13. Operator-Priorität
14. && und ||
15. NULL
16. IS_EMPTY und IS_NOT_EMPTY
17. IS_NULL und IS_NOT_NULL
18. Case Sensitivity
19. Nicht unterstützte Operatoren
20. Keine Arithmetik
21. Keine Standalone-Boolean-Expressions
22. Viele praktische Beispiele
    - Status ist READY
    - Status ist nicht READY
    - Status ist READY oder DONE
    - Status ist weder FAILED noch ERROR
    - Zwei Werte müssen vorhanden sein
    - Text enthält SUCCESS
    - Text enthält keinen ERROR
    - Ergebnis ist leer
    - Ergebnis ist nicht leer
    - Count ist 0
    - Count ist nicht 0
    - Feature ist aktiv und Status READY
    - Einer von zwei Prozessen läuft
    - Status ist PENDING oder PROCESSING
    - Antwort vorhanden und enthält READY
    - Komplexere Kombination
23. Verhalten bei normalen Steps
24. Verwendung durch Control-Flow-Engines
25. Bekannte Parser-Einschränkung
26. Best Practices
27. Troubleshooting
28. Kurzreferenz für KI / RAG

AGATE unterstützt über das Feld `condition` die bedingte Ausführung von Testschritten.

`condition` ist **keine Engine-spezifische Funktion**, sondern ein allgemeiner Bestandteil des AGATE DSL und kann grundsätzlich bei unterschiedlichen Engine-Typen verwendet

werden.

Beispiele:

```
- type: WAIT
  condition: "{B[enabled]} == 'true'"
  value: "1000"
```

```
- type: REST
  condition: "{B[status]} != 'FAILED'"
  op: EXEC
  command: demo.request
```

```
- type: CALL
  condition: "{R[userId]} != NULL"
  command: reusable.demo
```

Auch die LOOP-Engine verwendet dieselbe Condition-Syntax.

---

1. Condition Handling – Kompakte System-Referenz für KI-Modelle

```
AGATE CONDITION HANDLING

PURPOSE:
Controls conditional execution of AGATE DSL steps.

FIELD:
condition: "..."

GENERAL STEP SEMANTICS:

condition == true
-> execute step

condition == false
-> skip step

SKIPPED is not FAILED.

SUPPORTED BINARY OPERATORS:

Equality:
==
=
EQUALS

Inequality:
!=
<>
NOT_EQUALS

String:
CONTAINS

SUPPORTED LOGICAL OPERATORS:

AND
OR
NOT

ALIASES:

&& -> AND
|| -> OR

SUPPORTED GROUPING:

```

```
(...)

SUPPORTED UNARY OPERATORS:

IS_NULL
IS_NOT_NULL
IS_EMPTY
IS_NOT_EMPTY

RECOMMENDED VALUE SOURCES:

{B[name]}
{R[name]}

IMPORTANT:

Binary comparisons use STRING semantics.

NOT SUPPORTED:

<
>
<=
>=

numeric ordering
arithmetic expressions
standalone truthy expressions

SAFE EXAMPLES:

condition: "{B[count]} == 0"

condition: "{B[status]} == 'READY'"

condition: "{B[status]} != 'FAILED'"

condition: "{B[text]} CONTAINS 'SUCCESS'"

condition: "{R[value]} != NULL"

condition: "{B[a]} == 'X' AND {B[b]} == 'Y'"

condition: "{B[a]} == 'X' OR {B[a]} == 'Y'"

condition: "NOT {B[status]} == 'FAILED'"

condition: "({B[a]} == 'X' OR {B[b]} == 'Y') AND {B[c]} == 'Z'"

AI GENERATION RULE:

Prefer small and explicit conditions.

Do not generate:
<
>
<=
>=
arithmetic
standalone boolean/truthy expressions
```

---

2. Grundprinzip

Die Condition wird vor der Ausführung eines normalen Steps ausgewertet.

Beispiel:

```
- type: WAIT
```

```
  condition: "{B[enabled]} == 'true'"
  value: "2000"
```

Wenn:

```
enabled = true
```

ergibt die Condition:

```
true
```

und der WAIT-Step wird ausgeführt.

Wenn:

```
enabled = false
```

wird der Step übersprungen.

Typischer Log:

```
>>> SKIPPED (Condition false: false == 'true')
```

Ein übersprungener Step ist kein Testfehler.

---

3. Placeholder Resolution

Vor der eigentlichen Condition-Auswertung werden AGATE-Platzhalter aufgelöst.

Beispiel:

```
condition: "{B[status]} == 'READY'"
```

Wenn:

```
B[status] = READY
```

wird der Ausdruck logisch zu:

```
READY == READY
```

und ergibt:

```
true
```

Typische Quellen sind:

```
{B[...]}   Buffer / Runtime Value
{R[...]}   Reusable Parameter
```

---

4. Equality

Unterstützte Operatoren:

```
==
=
EQUALS
```

Diese Schreibweisen sind äquivalent.

Beispiele:

```
condition: "{B[status]} == 'READY'"
```

```
condition: "{B[status]} = 'READY'"
```

```
condition: "{B[status]} EQUALS 'READY'"
```

---

5. Inequality

Unterstützte Operatoren:

```
!=
<>
NOT_EQUALS
```

Beispiele:

```
condition: "{B[status]} != 'FAILED'"
```

```
condition: "{B[status]} <> 'FAILED'"
```

```
condition: "{B[status]} NOT_EQUALS 'FAILED'"
```

---

6. String-Vergleiche

Strings können mit einfachen oder doppelten Anführungszeichen verglichen werden.

Empfohlen:

```
condition: "{B[status]} == 'READY'"
```

Weitere Beispiele:

```
condition: "{B[result]} == 'SUCCESS'"
```

```
condition: "{R[action]} != 'DELETE'"
```

```
condition: "{B[response_code]} == '200'"
```

---

7. Numerisch aussehende Werte

Folgendes ist gültig:

```
condition: "{B[count]} == 0"
```

Aber dies ist **kein numerischer Vergleich**.

Der Evaluator behandelt beide Operanden als Strings.

Sinngemäß:

```
"0" == "0"
```

Das bedeutet:

```
condition: "{B[count]} == 0"
```

funktioniert.

Aber:

```
condition: "{B[count]} > 0"
```

funktioniert nicht.

---

8. CONTAINS

`CONTAINS` prüft, ob der rechte String im linken String enthalten ist.

Beispiele:

```
condition: "{B[message]} CONTAINS 'SUCCESS'"
```

```
condition: "{B[response_body]} CONTAINS 'READY'"
```

```
condition: "{B[log_output]} CONTAINS 'completed'"
```

```
condition: "{B[file_content]} CONTAINS 'expected-value'"
```

`CONTAINS` ist case-sensitive.

```
"SUCCESS" CONTAINS "SUCCESS"
-> true

"success" CONTAINS "SUCCESS"
-> false
```

---

9. AND

Mehrere Bedingungen können mit `AND` verbunden werden.

```
condition: "{B[status]} == 'READY' AND {B[result]} == 'OK'"
```

Weitere Beispiele:

```
condition: "{R[user]} != NULL AND {R[action]} != NULL"
```

```
condition: "{B[count]} == 0 AND {B[error]} != 'true'"
```

```
condition: "{B[state]} == 'PROCESSING' AND {B[retry]} == 'true'"
```

```
condition: "{R[enabled]} == 'true' AND {B[status]} != 'DONE'"
```

```
condition: "{B[response]} != NULL AND {B[response]} CONTAINS 'RUNNING'"
```

Alle Teilbedingungen müssen `true` sein.

---

10. OR

`OR` ist ebenfalls unterstützt.

```
condition: "{B[status]} == 'READY' OR {B[status]} == 'DONE'"
```

Weitere Beispiele:

```
condition: "{B[state]} == 'FAILED' OR {B[state]} == 'ERROR'"
```

```
condition: "{R[type]} == 'A' OR {R[type]} == 'B'"
```

```
condition: "{B[result]} == 'PENDING' OR {B[result]} == 'PROCESSING'"
```

```
condition: "{B[count]} == 0 OR {B[status]} == 'WAITING'"
```

Mindestens eine Teilbedingung muss `true` sein.

---

11. NOT

`NOT` negiert eine Condition.

Beispiele:

```
condition: "NOT {B[status]} == 'FAILED'"
```

```
condition: "NOT {B[result]} == 'ERROR'"
```

Mit Klammern:

```
condition: "NOT ({B[status]} == 'FAILED' OR {B[status]} == 'ERROR')"
```

---

12. Klammern

Runde Klammern können zur Gruppierung verwendet werden.

```
condition: "({B[a]} == 'X' OR {B[b]} == 'Y') AND {B[c]} == 'Z'"
```

Weitere Beispiele:

```
condition: "({B[status]} == 'READY' OR {B[status]} == 'DONE') AND
{B[error]} != 'true'"
```

```
condition: "{B[enabled]} == 'true' AND ({B[state]} == 'A' OR {B[state]}
== 'B')"
```

```
condition: "NOT ({B[state]} == 'FAILED' OR {B[state]} == 'ERROR') AND
{B[ready]} == 'true'"
```

---

13. Operator-Priorität

Die effektive Priorität der aktuellen Auswertung ist:

```
(...)
NOT
AND
OR
```

Beispiel:

```
A OR B AND C
```

wird interpretiert als:

```
A OR (B AND C)
```

Für komplexere Expressions sollten trotzdem explizite Klammern verwendet werden.

---

14. && und ||

Die Kurzformen:

```
&&
||
```

werden vor der Auswertung normalisiert.

Beispiel:

```
condition: "{B[a]} == '1' && {B[b]} == '2'"
```

entspricht:

```
condition: "{B[a]} == '1' AND {B[b]} == '2'"
```

Und:

```
condition: "{B[a]} == '1' || {B[b]} == '2'"
```

entspricht:

```
condition: "{B[a]} == '1' OR {B[b]} == '2'"
```

Für öffentliche Testfälle wird `AND` / `OR` empfohlen, da dies besser lesbar ist.

---

15. NULL

Für NULL-Vergleiche ist folgende Syntax etabliert:

```
condition: "{B[value]} == NULL"
```

```
condition: "{B[value]} != NULL"
```

Weitere Beispiele:

```
condition: "{R[userId]} != NULL"
```

```
condition: "{R[a]} != NULL AND {R[b]} != NULL"
```

```
condition: "{B[result]} == NULL OR {B[result]} == 'PENDING'"
```

Empfohlen ist:

```
NULL
```

ohne String-Quotes zu verwenden.

---

16. IS_EMPTY und IS_NOT_EMPTY

Der Evaluator unterstützt:

```
IS_EMPTY
IS_NOT_EMPTY
```

Beispiele:

```
condition: "{B[result]} IS_EMPTY"
```

```
condition: "{B[result]} IS_NOT_EMPTY"
```

```
condition: "{B[response]} IS_NOT_EMPTY AND {B[status]} == 'READY'"
```

---

17. IS_NULL und IS_NOT_NULL

Die Syntax ist im Evaluator vorhanden:

```
IS_NULL
IS_NOT_NULL
```

Beispiele:

```
condition: "{B[value]} IS_NULL"
```

```
condition: "{B[value]} IS_NOT_NULL"
```

Für die derzeitige DSL-Generierung wird jedoch bevorzugt:

```
condition: "{B[value]} == NULL"
```

bzw.

```
condition: "{B[value]} != NULL"
```

verwendet.

---

18. Case Sensitivity

Operatoren werden case-insensitive erkannt.

Beispielsweise:

```
AND
and
And
```

sind syntaktisch äquivalent.

Dasselbe gilt für:

```
EQUALS
equals
```

```
Equals
```

Die verglichenen Werte selbst sind jedoch case-sensitive.

```
READY == READY
-> true

ready == READY
-> false
```

---

19. Nicht unterstützte Operatoren

Nicht unterstützt:

```
<
>
<=
>=
```

Daher nicht verwenden:

```
condition: "{B[count]} > 5"
```

```
condition: "{B[count]} < 10"
```

```
condition: "{B[count]} >= 3"
```

```
condition: "{B[count]} <= 20"
```

---

20. Keine Arithmetik

Nicht unterstützt:

```
condition: "{B[counter]} + 1 == 5"
```

```
condition: "{B[value]} * 2 == 10"
```

```
condition: "{B[value]} / 2 == 4"
```

Der ConditionEvaluator ist keine mathematische Expression Engine.

---

21. Keine Standalone-Boolean-Expressions

Nicht verwenden:

```
condition: "true"
```

Nicht verwenden:

```
condition: "{B[enabled]}"
```

Stattdessen explizit vergleichen:

```
condition: "{B[enabled]} == 'true'"
```

---

22. Viele praktische Beispiele

Status ist READY

```
condition: "{B[status]} == 'READY'"
```

Status ist nicht READY

```
condition: "{B[status]} != 'READY'"
```

Status ist READY oder DONE

```
condition: "{B[status]} == 'READY' OR {B[status]} == 'DONE'"
```

Status ist weder FAILED noch ERROR

```
condition: "NOT ({B[status]} == 'FAILED' OR {B[status]} == 'ERROR')"
```

Zwei Werte müssen vorhanden sein

```
condition: "{R[userId]} != NULL AND {R[action]} != NULL"
```

Text enthält SUCCESS

```
condition: "{B[text]} CONTAINS 'SUCCESS'"
```

Text enthält keinen ERROR

```
condition: "NOT {B[text]} CONTAINS 'ERROR'"
```

Ergebnis ist leer

```
condition: "{B[result]} IS_EMPTY"
```

Ergebnis ist nicht leer

```
condition: "{B[result]} IS_NOT_EMPTY"
```

Count ist 0

```
condition: "{B[count]} == 0"
```

Count ist nicht 0

```
condition: "{B[count]} != 0"
```

Feature ist aktiv und Status READY

```
condition: "{B[enabled]} == 'true' AND {B[status]} == 'READY'"
```

Einer von zwei Prozessen läuft

```
condition: "{B[processA]} == 'RUNNING' OR {B[processB]} == 'RUNNING'"
```

Status ist PENDING oder PROCESSING

```
condition: "{B[status]} == 'PENDING' OR {B[status]} == 'PROCESSING'"
```

Antwort vorhanden und enthält READY

```
condition: "{B[response]} != NULL AND {B[response]} CONTAINS 'READY'"
```

Komplexere Kombination

```
condition: "({B[state]} == 'READY' OR {B[state]} == 'DONE') AND
{B[error]} != 'true'"
```

---

23. Verhalten bei normalen Steps

Für normale Steps:

```
condition true
-> Step ausführen

condition false
-> Step SKIPPED
```

Beispiel:

```
- type: WAIT
  condition: "{B[enabled]} == 'true'"
  value: "1000"
```

oder:

```
- type: CALL
  condition: "{R[userId]} != NULL"
  command: reusable.demo
```

---

24. Verwendung durch Control-Flow-Engines

Control-Flow-Komponenten wie LOOP verwenden dieselbe zentrale Condition-Syntax.

Die konkrete Bedeutung von `true` und `false` kann dabei Engine-spezifisch sein.

Bei normalen Steps:

```
true  -> execute
false -> SKIPPED
```

Bei LOOP:

```
true  -> another iteration
false -> LOOP end
```

Die LOOP-spezifische Semantik ist im separaten Dokument **LOOP Engine** beschrieben.

---

25. Bekannte Parser-Einschränkung

Die aktuelle Logik berücksichtigt beim Aufteilen von `AND` und `OR` Klammerung, aber keine String-Quotes.

Daher sollten Stringwerte möglichst nicht selbst:

```
 AND
```

oder:

```
 OR
```

enthalten.

Nicht empfohlen:

```
condition: "{B[text]} == 'A OR B'"
```

---

26. Best Practices

Conditions möglichst klein halten.

Bevorzugt:

```
condition: "{B[status]} == 'READY'"
```

oder:

```
condition: "{B[status]} != 'FAILED' AND {B[result]} == 'OK'"
```

Für technisch erzeugte Werte bevorzugt:

```
ENGINE
   ↓
BUFFER
   ↓
CONDITION
```

Dadurch bleibt Condition Handling unabhängig von der konkreten technischen Engine.

---

27. Troubleshooting

Wenn eine Condition unerwartet `false` ergibt:

1. Prüfen, ob alle Placeholder aufgelöst wurden.
2. Prüfen, welcher tatsächliche Buffer-Wert vorhanden ist.
3. Groß-/Kleinschreibung prüfen.
4. Quotes prüfen.
5. NULL prüfen.
6. Sicherstellen, dass nur unterstützte Operatoren verwendet werden.
7. Prüfen, ob der benötigte Wert bereits existiert.

---

28. Kurzreferenz für KI / RAG

```
AGATE CONDITION HANDLING

GLOBAL DSL FEATURE:
Condition is not tied to one engine.

FIELD:
condition
```

```
NORMAL STEP:
true  -> execute
false -> SKIPPED

SUPPORTED:

==
=
EQUALS

!=
<>
NOT_EQUALS

CONTAINS

AND
OR
NOT

&&
||

(...)

IS_NULL
IS_NOT_NULL
IS_EMPTY
IS_NOT_EMPTY

RECOMMENDED NULL:
== NULL
!= NULL

VALUES:
binary comparisons are string based

CASE:
operator names case-insensitive
values case-sensitive

NOT SUPPORTED:
<
>
<=
>=
arithmetic
numeric ordering
standalone truthy expressions

SAFE:
{B[status]} == 'READY'
{B[status]} != 'FAILED'
{B[count]} == 0
{B[text]} CONTAINS 'SUCCESS'
{R[value]} != NULL
{B[a]} == 'X' AND {B[b]} == 'Y'
{B[a]} == 'X' OR {B[a]} == 'Y'
NOT {B[state]} == 'FAILED'
({B[a]} == 'X' OR {B[b]} == 'Y') AND {B[c]} == 'Z'

PREFERRED DESIGN:
ENGINE
-> BUFFER
-> CONDITION

AI RULE:
Never invent unsupported numerical comparison operators.
```